EXPERIMENT NO. 02– Shortest Path Algorithms



Name: Dibyanshu Shekhar Dey
Roll Number: 25EC64R23

Design and Simulation Lab: Communication Networks
Department of Electronics & Electrical Communication Engineering
Indian Institute of Technology Kharagpur

AIM: Explain for each of the following using formal description and example for each of them and compare them:

- Dijkstra's algorithm
- Bellman Ford Algorithm
- Floyd-Warshall algorithm

Puzzle : Explain the impact on Dijkstra Algorithm if the edge of a graph is replaced and it is known beforehand which edge has been replaced.

INTRODUCTION:

Graph algorithms play a fundamental role in computer science and engineering, with applications ranging from network routing and transportation systems to operating systems and communication networks. Among these, shortest path algorithms are used to determine the minimum-cost paths between vertices in a weighted graph, where costs may represent distance, time, or any other metric. Efficient computation of shortest paths becomes essential for solving real-world problems such as traffic navigation, packet routing, and resource optimization.

This experiment focuses on the study and implementation of three classical shortest-path algorithms: Dijkstra's algorithm, the Bellman–Ford algorithm, and the Floyd–Warshall algorithm.

THEORY:

Dijkstra's Algorithm:

The Dijkstra algorithm is a classical greedy algorithm used to solve the single-source shortest-path problem on a weighted graph. Given a graph $G = (V, E)$ and a source vertex s, it computes the minimum-cost paths from s to all other vertices of the graph. The algorithm assumes that all edge weights are nonnegative, i.e., $w(u, v) \geq 0$ for every edge $(u, v) \in E$, which guaranties correctness [1]. It is widely used in practical applications, such as network routing protocols [2].

It maintains a set of vertices whose final shortest-path distances from the source have already been determined. At each step, it selects the unvisited vertex with the minimum tentative distance, adds it to the set of finalized vertices, and relaxes all outgoing edges from that vertex to update neighboring distances if a shorter path is found. This process continues until all vertices are processed [3].

Algorithm 1 Dijkstra's Algorithm for Shortest Path [1]

Select the source vertex s
Initialize the set $N \leftarrow \{s\}$
Set distance $d[s] \leftarrow 0$
for each vertex $v \neq s$ **do**
    Set distance $d[v] \leftarrow \infty$
end for
while there exist vertices $v \notin N$ **do**
    Select a vertex $u \notin N$ with minimum distance $d[u]$
    Add vertex u to N
    for each vertex $v \notin N$ adjacent to u **do**
        if $d[u] + w(u, v) < d[v]$ **then**
            Set $d[v] \leftarrow d[u] + w(u, v)$
        end if
    end for
end while

Algorithmic Progression for given graph used in the program: //

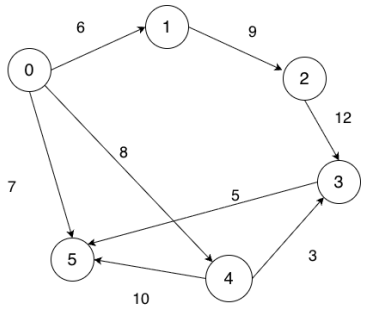


Figure 1: Initial Graph (with edge weights and vertices)

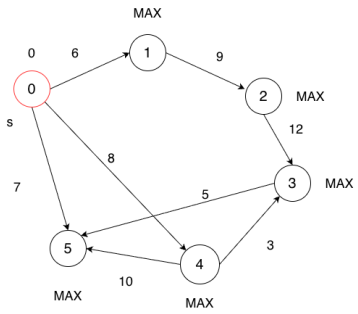


Figure 2: Initialize the graph at vertex 0

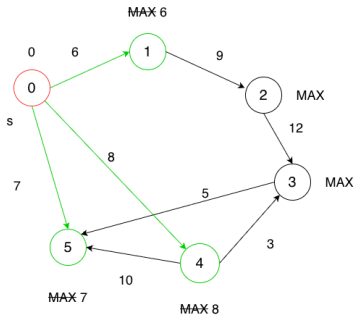


Figure 3: Now Algorithm searches for all the adjacent weights and relaxes the vertices with the edge weights (see Vertices 0, 4, 5)

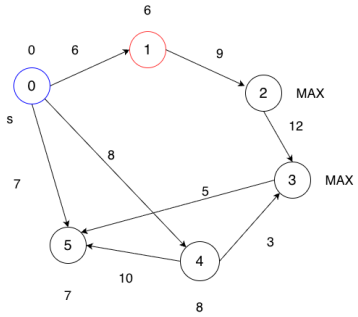


Figure 4: Now Algorithm selects the minimum weight (Vertex 1 with weight 6) and marks it.

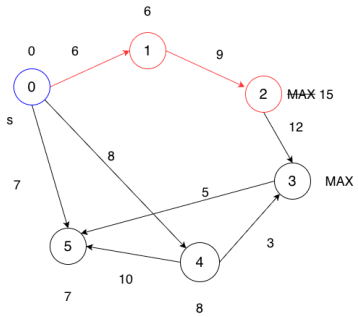


Figure 5: Now algorithm takes the minimum weight in the list this time 6 and searches for adjacent weights and updates if the distance is less (here it updates vertex 2 with weight 15.

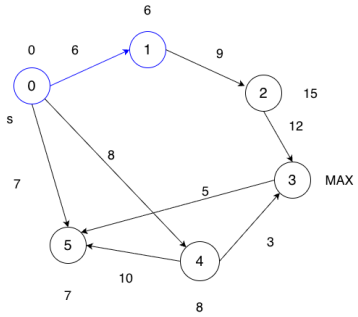


Figure 6: The algorithm has updated list with visited vertices and weights updated in the vertices.

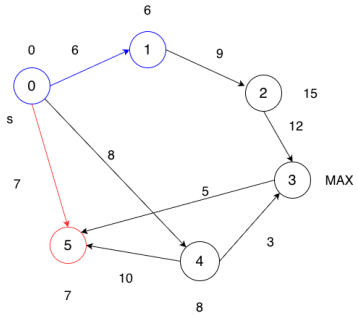


Figure 7: Now algorithm takes the minimum weighted vertex and searches for the adjacent vertices to update the weights.

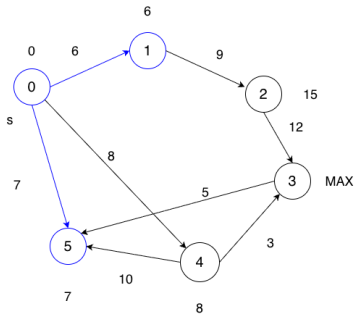


Figure 8: As there are no outgoing edges from the vertex 5, the vertex is marked as visited and the weight is removed from the list.

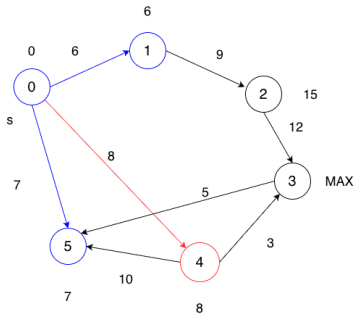


Figure 9: Again algorithm takes the minimum weighted vertex (vertex 4 with weight 8

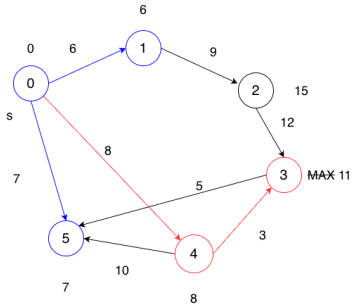


Figure 10: It searches for the adjacent vertices (here vertex 3 and updates the weight from Max to 11.

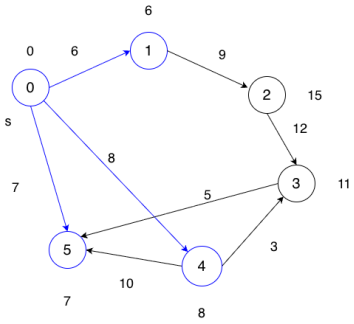


Figure 11: The algorithm removes the minimum weight ans vertex and marks it as visited.

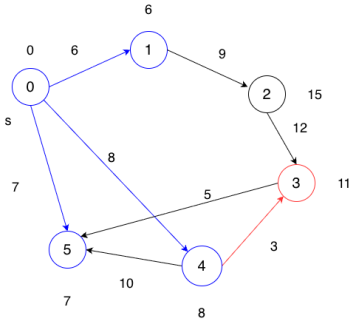


Figure 12: Vertex 3 is checked (weights in the list) from vertex 4 and updated. The algorithm checks the outgoing edges to check if the minimum distance is less than the updated vertices weights. If it is not then it is marked as visited and the weight is popped from the list to be updated.

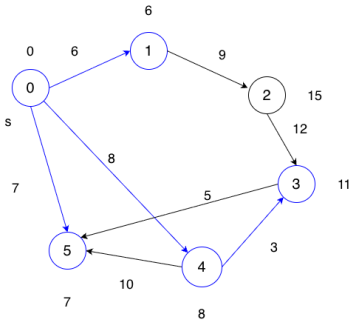


Figure 13: Updated graph with weighed vertices. Only minimum weight left is 15.

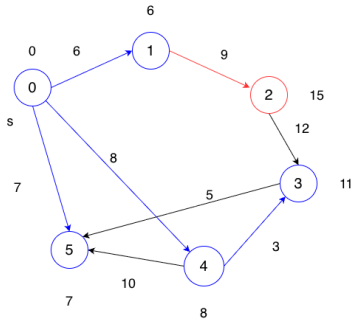


Figure 14: It is from vertex 1 and then it is updated.

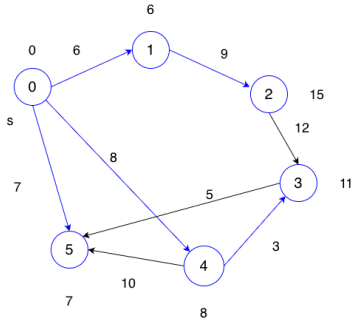


Figure 15: Final vertices with updated weights from the source vertex 0.

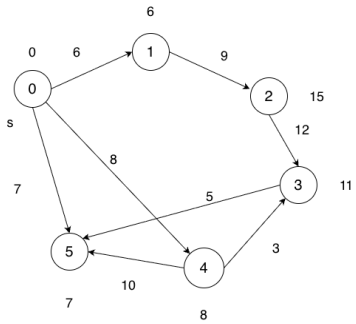


Figure 16: Final Graph

Bellman Ford's Algorithm:
The Bellman–Ford algorithm helps in solving the single-source shortest-paths problem in weighted directed graphs where edge weights may be negative. Unlike Dijkstra's algorithm, it can correctly handle negative-weight edges, but it does not produce a valid shortest-path solution if a negative-weight cycle is reachable from the source (see the illustrated example), since traversing such a cycle can indefinitely reduce the path cost. The algorithm addresses this by not only computing shortest-path distances but also explicitly detecting the presence of negative-weight cycles and reporting when no finite solution exists[1].

The algorithm works by repeatedly relaxing all edges in the graph, progressively decreasing the estimated distance to each vertex. Specifically, it performs $|V| - 1$ passes over all edges, where each pass allows shortest paths with one additional edge to be discovered [1], [4]. After these

passes, a final scan of the edges is used to check whether any distance can still be reduced; if so, a negative-weight cycle exists. When the graph is represented using adjacency lists, the Bellman–Ford algorithm runs in $O(VE)$ time in typical cases, making it slower than Dijkstra's algorithm for graphs with only non-negative weights, but essential when negative edges or cycle detection are required [1], [5].

Algorithm 2 Bellman–Ford Algorithm for Shortest Path [1]

```
Select the source vertex s
Set distance d[s] ← 0
for each vertex v ≠ s do
    Set distance d[v] ← ∞
end for
for i = 1 to |V| − 1 do
    for each edge (u, v) ∈ E do
        if d[u] + w(u, v) < d[v] then
            Set d[v] ← d[u] + w(u, v)
        end if
    end for
end for
for each edge (u, v) ∈ E do
    if d[u] + w(u, v) < d[v] then
        Report negative weight cycle
        EXIT
    end if
end for
```

Algorithmic progression of Bellman Ford Algorithm

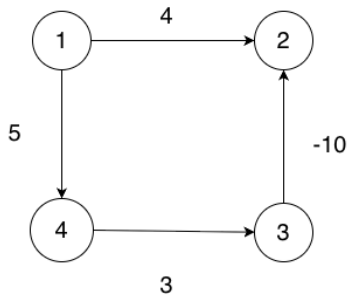


Figure 17: Inital Graph

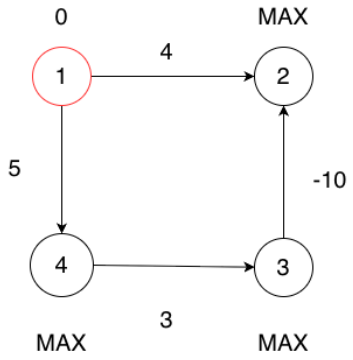


Figure 18: Initialization with vertex 0

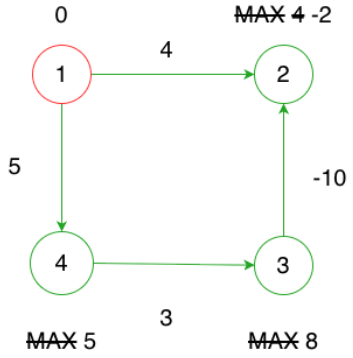


Figure 19: First path traversal across the edges (directed paths) if $d[u] + w(u, v) < d[v]$, then $d[v]$ is updated (see ~~MAX~~ updated)

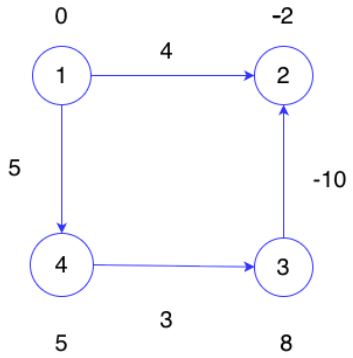


Figure 20: Second time path traversal.

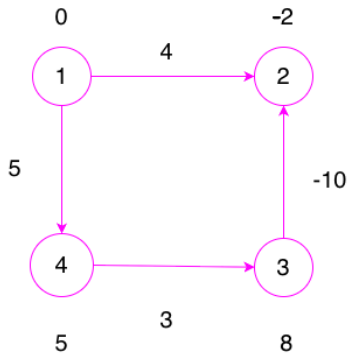


Figure 21: Third time path traversal (Last as per the algorithm).

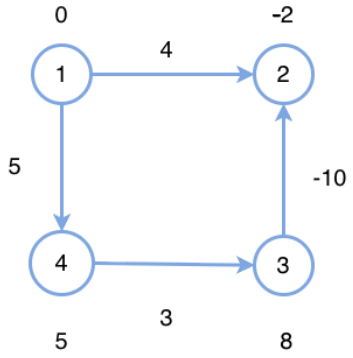


Figure 22: A Hypothetical path traversal to check if further relaxation is possible.

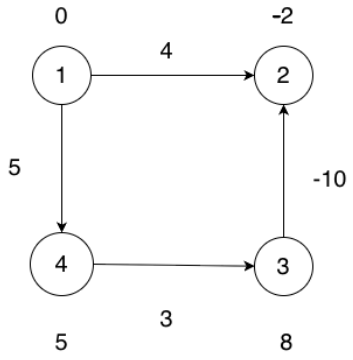


Figure 23: Final Graph

Drawback of Bellman-Ford

The following figures illustrates as why Bellman-Ford fails for negative cycles [5].

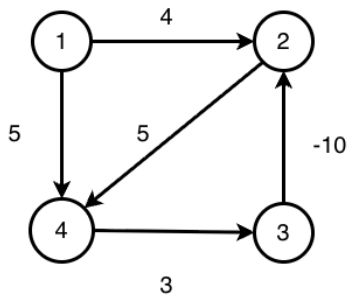


Figure 24: An example graph to check the drawback of Bellman-Ford

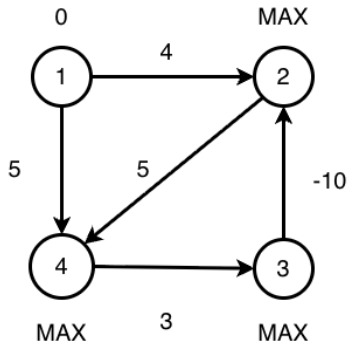


Figure 25: Graph initialization with 0 weight attached to vertex 0 and all others MAX

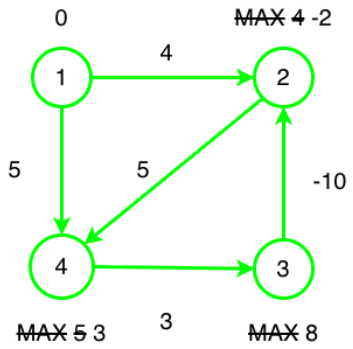


Figure 26: First path traversal across all the edges and weight updation of the vertices as per the algorithm ($d[u] + w(u, v) < d[v]$, then $d[v]$ is updated)

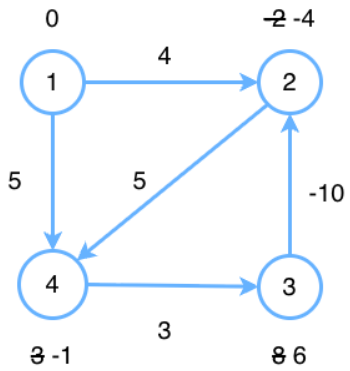


Figure 27: Second path traversal

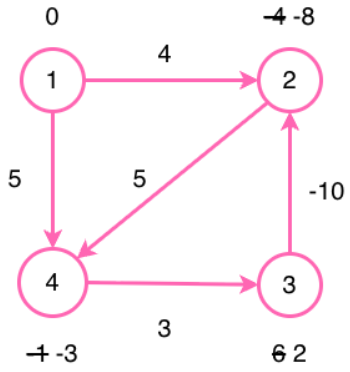


Figure 28: Third path traversal (Here the number of steps taken by the code as per algorithm is completed). However there might be more traversals.

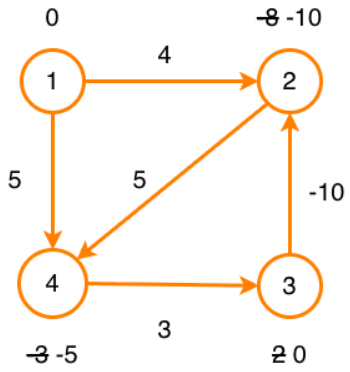


Figure 29: On the fourth path traversal , the vertices 2 , 4, 3 are further updated. And if the code continues without any check, there will be infinite possibilities.

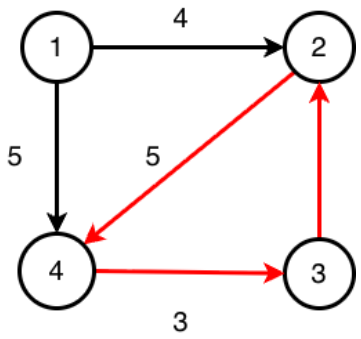


Figure 30: This happens because of the negative cycle taht forsm between 4, 3 and 2.

Floyd Warshall Algorithm:
Floyd–Warshall is a dynamic programming algorithm which is used to solve the all-pairs shortest-paths problem in a weighted directed graph. It computes the shortest distances between every pair of vertices by allowing intermediate vertices in the paths one at a time. The algorithm runs in $\Theta(|V|^3)$ time and can handle graphs with negative-weight edges, given that no negative-weight cycles are present [1]. The core idea is to iteratively improve an initial distance matrix by checking whether a path from vertex i to vertex j can be lessened by passing through an intermediate vertex k [6].

The algorithm begins by constructing an initial distance matrix in which each entry represents the direct edge weight between two vertices, infinity (or a maximum weight associated) if no direct edge exists, and zero along the diagonal [7].
Then, for each vertex chosen as an intermediate pivot, the algorithm updates all pairs of distances using the recurrence relation $d[i][j] = \min(d[i][j],\ d[i][k] + d[k][j])$. By repeating this process for all vertices as possible intermediates, the final matrix produced contains the shortest path distances between every pair of vertices in the graph [1], [8].

Algorithm 3 Floyd–Warshall Algorithm[1]

Let dist be a $|V| \times |V|$ array initialized to ∞
for each edge (u, v) **do**
    dist$[u][v] \leftarrow w(u, v)$
end for
for each vertex v **do**
    dist$[v][v] \leftarrow 0$
end for
for $k = 1$ to $|V|$ **do**
    for $i = 1$ to $|V|$ **do**
        for $j = 1$ to $|V|$ **do**
            if dist$[i][j] >$ dist$[i][k] +$ dist$[k][j]$ **then**
                dist$[i][j] \leftarrow$ dist$[i][k] +$ dist$[k][j]$
            end if
        end for
    end for
end for

The following is a graph that is used to analyse the Floyd-Warshall Algorithm.

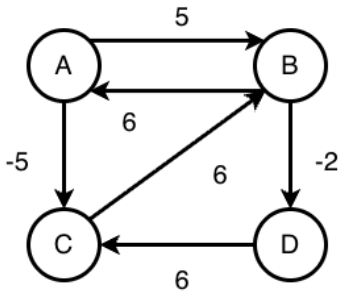


Figure 31: This happens because of the negative cycle taht forsm between 4, 3 and 2.

Step 1: - When the weights are not initilised, only the edge weights are added.

$$d^{(initial)} = \begin{pmatrix} 0 & 5 & -5 & 0 \\ 4 & 0 & 0 & -2 \\ 0 & 7 & 0 & 0 \\ 0 & 0 & 6 & 0 \end{pmatrix}$$

Step 2: - Here no direct paths are added as ∞, whereas the diagonals vertices are reinitialized as 0.

$$d^{(0)} = \begin{pmatrix} 0 & 5 & -5 & \infty \\ 4 & 0 & \infty & -2 \\ \infty & 7 & 0 & \infty \\ \infty & \infty & 6 & 0 \end{pmatrix}$$

Step 3: - Algorithmic traversal when vertex = A. In red, the weights are the updated weights in the current iteration, the blue weights will be the weights updated from previous iteration.

$$d^{(a)} = \begin{pmatrix} 0 & 5 & -5 & \infty \\ 4 & 0 & -1 & -2 \\ \infty & 7 & 0 & \infty \\ \infty & \infty & 6 & 0 \end{pmatrix}$$

Step 4: - Algorithmic traversal when vertex = B.

$$d^{(b)} = \begin{pmatrix} 0 & 5 & -5 & 3 \\ 4 & 0 & -1 & -2 \\ 11 & 7 & 0 & 5 \\ \infty & \infty & 6 & 0 \end{pmatrix}$$

Step 5: - Algorithmic traversal when vertex = C.

$$d^{(c)} = \begin{pmatrix} 0 & 2 & -5 & 0 \\ 4 & 0 & -1 & -2 \\ 11 & 7 & 0 & 5 \\ 17 & 13 & 6 & 0 \end{pmatrix}$$

Step 6: - Algorithmic traversal when vertex = D.

$$d^{(d)} = \begin{pmatrix} 0 & 2 & -5 & 0 \\ 4 & 0 & -1 & -2 \\ 11 & 7 & 0 & 5 \\ 17 & 13 & 6 & 0 \end{pmatrix}$$

Result: Final graph matrix.

$$d^{(final)} = \begin{pmatrix} 0 & 2 & -5 & 0 \\ 4 & 0 & -1 & -2 \\ 11 & 7 & 0 & 5 \\ 17 & 13 & 6 & 0 \end{pmatrix}$$

RESULTS:
Output of Dijkstra's Algorithm

```
Adjacency Matrix Representation
0 1 0 0 0 0 0 0 8 3
1 0 4 0 0 0 0 2 0 0
0 4 0 -6 2 0 0 0 0 0
0 0 -6 0 7 0 0 0 -3 0
0 0 2 7 0 -10 0 0 0 0
0 0 0 0 -10 0 7 -5 0 0
0 0 0 0 0 7 0 4 9 0
0 2 0 0 0 -5 4 0 2 0
8 0 0 -3 0 0 9 2 0 -4
3 0 0 0 0 0 0 0 -4 0
Dijkstra shortest paths are:
0 ---> 0
1 ---> 1
2 ---> -10
3 ---> -4
4 ---> -8
5 ---> -18
6 ---> -19
7 ---> -23
8 ---> -1
9 ---> 3
```

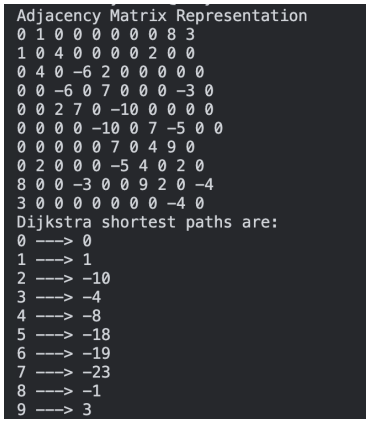


Figure 32: Dijkstra's Algorithm Result in compiler

Output of Bellman-Ford Algorithm

```
Adjacency Matrix Representation
0 1 0 0 0 0 0 0 0 3
0 0 5 0 0 0 0 2 0 0
0 4 0 -6 2 0 0 0 0 0
0 0 0 0 0 0 0 0 -3 0
0 0 0 7 0 -10 0 0 0 0
0 0 0 0 0 0 7 0 0 0
0 0 0 0 0 0 0 4 0 0
0 0 0 0 0 -5 0 0 2 0
8 0 0 0 0 0 9 0 0 0
0 0 0 0 0 0 0 0 -4 0
Bellman-Ford shortest paths are:
0 ---> 0
1 ---> 1
2 ---> 6
3 ---> 0
4 ---> 8
5 ---> -2
6 ---> 5
7 ---> 3
8 ---> -3
9 ---> 3
```

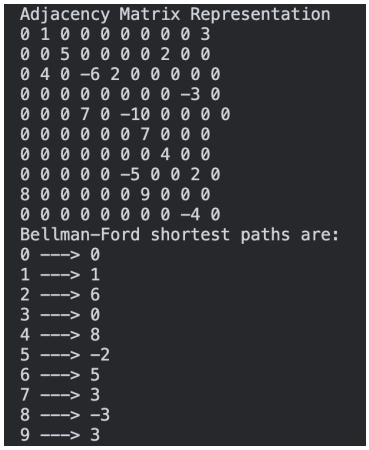


Figure 33: Bellman-Ford Algorithm Result in compiler

Output of Floyd-Warshall Algorithm

```
Adjacency Matrix Representation
0 5 -5 0
4 0 0 -2
0 7 0 0
0 0 6 0
The matrix after adding infinity is
0 5 -5 999
4 0 999 -2
999 7 0 999
999 999 6 0
Step 1
----------------
0 5 -5 999
4 0 -1 -2
999 7 0 999
999 999 6 0
----------------
Step 2
----------------
0 5 -5 3
4 0 -1 -2
11 7 0 5
999 999 6 0
----------------
Step 3
----------------
0 2 -5 0
4 0 -1 -2
11 7 0 5
17 13 6 0
----------------
Step 4
```

```
----------------
0 2 -5 0
4 0 -1 -2
11 7 0 5
17 13 6 0
----------------
The matrix after adding Alg implementation is
0 2 -5 0
4 0 -1 -2
11 7 0 5
17 13 6 0
```

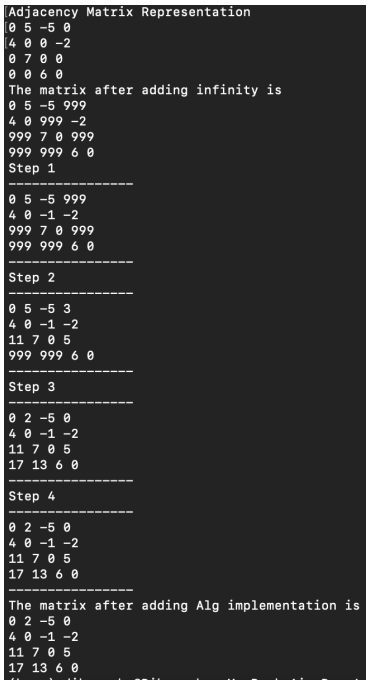


Figure 34: Floyd-Warshall Algorithm Result in compiler

DISCUSSION:

Table 1: Comparison of Shortest Path Algorithms

Feature	**Dijkstra's**	**Bellman–Ford**	**Floyd–Warshall**
Source Type	Single-source	Single-source	All-pairs
Negative Edge Weights	Not allowed	Allowed	Allowed
Negative Cycle Detection	No	Yes	No
Algorithmic Technique	Greedy	Dynamic Programming	Dynamic Programming
Time Complexity	$O((V+E)\log V)$	$O(VE)$	$O(V^3)$

Puzzle: Given a graph, if the edge of the graph is changed, and we have a prior knowledge about it. Explain the use of Dijkstra' Algorithm in this case and how it can be optimized without the use of Dijkstra.

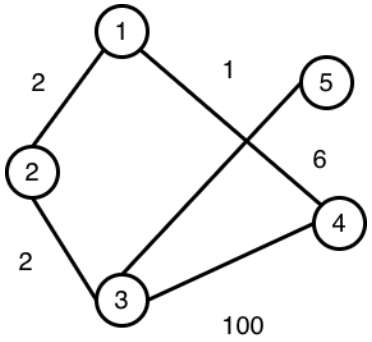


Figure 35: Graph under consideration

Proposed Solution: Case 1: If the edge affected is the part of shortest path, and if the edge weight is less than the previous weight assigned, then there is no problem as Dijkstra will not be used.

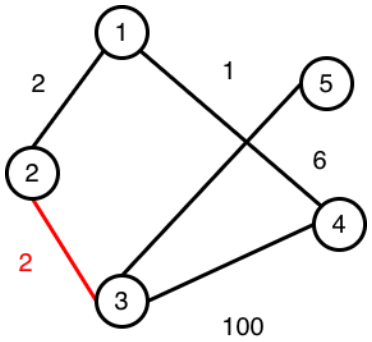


Figure 36: Graph after weight change

Case 2: If the edge affected is the part of shortest path, and if the edge weight is greater than the previous weight assigned, then only the relaxation from effective vertices can be considered and different algorithms may be used to choose the next shortest path and compare them along the way.

ACKNOWLEDGEMENT:
I would like to share my thanks to Prof. Amitalok J Budkuley for explaining the context of the algorithms. I would like to extend my thanks to TAs Anuj Meena and Siva Sankar Thummala for questioning the fundamentals of the experiment and helping throughout along with the prblem description. I would like to thank my colleagues Mayank Singh, Saptarshi Bhattacharjee and Soumava Biswas for collaborating in lab throughout the experiment.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd. Cambridge, MA, USA: MIT Press, 2009.

[2] R. Thareja, *Data Structures Using C*, 2nd ed. New Delhi, India: Oxford University Press, 2014.

[3] A. Bari. “Dijkstra algorithm – single source shortest path – greedy method.” [Online; accessed 1-February-2026], YouTube. [Online]. Available: https://www.youtube.com/watch?v=XB4MIexjvY0.

[4] W3Schools. “Bellman–ford algorithm.” [Online; accessed 1-February-2026]. [Online]. Available: https://www.w3schools.com/dsa/dsa_algo_graphs_bellmanford.php.

[5] A. Bari. “Bellman ford algorithm – single source shortest path – dynamic programming.” [Online; accessed 1-February-2026], YouTube. [Online]. Available: https://www.youtube.com/watch?v=FtN3BYH2Zes.

[6] W. contributors. “Floyd–warshall algorithm.” [Online; accessed 1-February-2026]. [Online]. Available: https://en.wikipedia.org/wiki/Floyd%E2%80%93Warshall_algorithm.

[7] TutorialsPoint. “Floyd warshall algorithm.” [Online; accessed 1-February-2026]. [Online]. Available: https://www.tutorialspoint.com/data_structures_algorithms/floyd_warshall_algorithm.htm.

[8] CP-Algorithms. “All pairs shortest path — floyd–warshall algorithm.” [Online; accessed 1-February-2026]. [Online]. Available: https://cp-algorithms.com/graph/all-pair-shortest-path-floyd-warshall.html.

APPENDIX (CODE):

Djikstra’s Algorithm in C

```c
#include <stdio.h>
#include <stdbool.h>

#define V 10
#define MAX 999

int minDist(int distArr[], bool vertArr[]){
    int minm = MAX;
    int index;
    for (int i = 0; i < V; i++){
        if(distArr[i] <= minm && vertArr[i] == false){
            minm = distArr[i];
            index = i;
        }
    }
    return  index;
}
void Dijkstra(int mat[V][V], int src){
    int distArr[V];
    bool vertArr[V];
    int min;
    for(int i = 0; i < V; i++){
        distArr[i] = MAX;
        vertArr[i] = false;
    }

    distArr[src] = 0;
    for(int i = 0;  i < V; i++){
        min = minDist(distArr, vertArr);
```

```
        vertArr[min] = true;
        for(int j = 0; j < V; j++){
            if(!vertArr[j] && mat[min][j] && distArr[min] != MAX && distArr
                [min] + mat[min][j] < distArr[j]){
                distArr[j] = distArr[min] + mat[min][j];
            }
        }
    }
    printf("Dijkstra shortest paths are:\n");
    for(int i = 0; i < V; i++){
        printf("%d ---> %d\n", i, distArr[i]);
    }

}

void addEdge(int mat[V][V], int i, int j, int w){
    mat[i][j] = w;
    mat[j][i] = w;
}


// to display matrix
void displayMatrix(int mat[V][V]){
    for (int i = 0; i < V; i++)
    {
        /* code */
        for (int j = 0; j < V; j++)
        {
            /* code */
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }
}

int main() {
    /* code */
    int mat[V][V] = {0};
    addEdge(mat, 0, 1, 1);
    addEdge(mat, 0, 9, 3);
    addEdge(mat, 1, 2, 5);
    addEdge(mat, 1, 7, 2);
    addEdge(mat, 2, 1, 4);
    addEdge(mat, 2, 3, -6);
    addEdge(mat, 2, 4, 2);
    addEdge(mat, 3, 8, -3);
    addEdge(mat, 4, 3, 7);
    addEdge(mat, 4, 5, -10);
    addEdge(mat, 5, 6, 7);
    addEdge(mat, 6, 7, 4);
    addEdge(mat, 7, 5, -5);
    addEdge(mat, 7, 8, 2);
    addEdge(mat, 8, 6, 9);
    addEdge(mat, 8, 0, 8);
    addEdge(mat, 9, 8, -4 );
    printf("Adjacency Matrix Representation\n");
    displayMatrix(mat);

```

```
    Dijkstra(mat, 0);
    return 0;
}
```

Bellman-Ford's Algorithm in C

```
    #include <stdio.h>
#include <stdbool.h>

#define V 10
#define MAX 999

void BellmanFord(int mat[V][V], int src){
    int distArr[V];

    // Step 1: Initialize distances
    for(int i = 0; i < V; i++){
        distArr[i] = MAX;
    }
    distArr[src] = 0;

    // Step 2: Relax all edges V-1 times
    for(int k = 0; k < V - 1; k++){
        for(int i = 0; i < V; i++){
            for(int j = 0; j < V; j++){
                if(mat[i][j] != 0 && distArr[i] != MAX &&
                   distArr[i] + mat[i][j] < distArr[j]){
                    distArr[j] = distArr[i] + mat[i][j];
                }
            }
        }
    }

    // Step 3: Check for negative weight cycles
    for(int i = 0; i < V; i++){
        for(int j = 0; j < V; j++){
            if(mat[i][j] != 0 && distArr[i] != MAX &&
               distArr[i] + mat[i][j] < distArr[j]){
                printf("Graph contains a negative weight cycle\n");
                return;
            }
        }
    }

    // Print result
    printf("Bellman-Ford shortest paths are:\n");
    for(int i = 0; i < V; i++){
        printf("%d ---> %d\n", i, distArr[i]);
    }
}

void addEdge(int mat[V][V], int i, int j, int w){
    mat[i][j] = w;
    // For undirected graph, keep this:
    //mat[j][i] = w;
}

// Display matrix
void displayMatrix(int mat[V][V]){
```

```
    for(int i = 0; i < V; i++){
        for(int j = 0; j < V; j++){
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }
}

int main(){
    int mat[V][V] = {0};

    addEdge(mat, 0, 1, 1);
    addEdge(mat, 0, 9, 3);
    addEdge(mat, 1, 2, 5);
    addEdge(mat, 1, 7, 2);
    addEdge(mat, 2, 1, 4);
    addEdge(mat, 2, 3, -6);
    addEdge(mat, 2, 4, 2);
    addEdge(mat, 3, 8, -3);
    addEdge(mat, 4, 3, 7);
    addEdge(mat, 4, 5, -10);
    addEdge(mat, 5, 6, 7);
    addEdge(mat, 6, 7, 4);
    addEdge(mat, 7, 5, -5);
    addEdge(mat, 7, 8, 2);
    addEdge(mat, 8, 6, 9);
    addEdge(mat, 8, 0, 8);
    addEdge(mat, 9, 8, -4 );


    printf("Adjacency Matrix Representation\n");
    displayMatrix(mat);

    BellmanFord(mat, 0);
    return 0;
}
```

Floyd-Warshall's Algorithm in C

```

#include <stdio.h>
#include <stdbool.h>
#include <math.h>

#define V 4
#define MAX 999

void floydWarshall(int mat[V][V]);
int minDist(int distArr[], bool vertArr[]);
void addEdge(int mat[V][V], int i, int j, int w);
void displayMatrix(int mat[V][V]);
int min(int, int);

int min(int i, int j){
    int min = 0;
    if (i < j){
        min = i;
    }
    else{
```

```
        min = j;
    }
    return min;
}

int minDist(int distArr[], bool vertArr[]){
    int minm = MAX;
    int index;
    for (int i = 0; i < V; i++){
        if(distArr[i] <= minm && vertArr[i] == false){
            minm = distArr[i];
            index = i;
        }
    }
    return  index;
}

void floydWarshall(int mat[V][V]){
    for (int i = 0; i < V; i++)
    {
        /* code */
        for (int j = 0; j < V; j++)
        {
            /* code */
            if(i == j){
                mat[i][j] = 0;
            }
            else{
                if(i != j && mat[i][j] != 0){
                    mat[i][j] = mat[i][j];
                }
                else{
                    mat[i][j] = MAX;
                }
            }
        }
    }
    printf("The matrix after adding infinity is\n");
    displayMatrix(mat);

    for(int k = 0; k < V; k++){
        for (int i = 0; i < V; i++){
            for(int j = 0; j < V; j++){
                if (mat[i][j]> mat[i][k] + mat[k][j]){ // (mat [i][k] !=
                    MAX && mat[k][j] != MAX)  { // && (mat [k][j] != MAX &&
                    mat[i][k] != MAX))
                    mat[i][j] = min(mat[i][j], mat[i][k] + mat[k][j]);
                    //mat[i][j] = mat[i][k] + mat[k][j];
                }
                else{
                    mat[i][j] = mat[i][j];
                }
            }

        }
        printf("Step %d\n", k+1);
        printf("----------------\n");
        displayMatrix(mat);
```

```
        printf("----------------\n");
    }

    printf("The matrix after adding Alg implementation is\n");
    displayMatrix(mat);


}
void addEdge(int mat[V][V], int i, int j, int w){
    mat[i][j] = w;
    //mat[j][i] = w;
}


// to display matrix
void displayMatrix(int mat[V][V]){
    for (int i = 0; i < V; i++)
    {
        /* code */
        for (int j = 0; j < V; j++)
        {
            /* code */
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }
}

int main() {
    /* code */
    int mat[V][V] = {0};
    addEdge(mat, 0, 1, 5);
    addEdge(mat, 0, 2, -5);
    addEdge(mat, 1, 0, 4);
    addEdge(mat, 2, 1, 7);
    addEdge(mat, 3, 2, 6);
    addEdge(mat, 1, 3, -2);
    //addEdge(mat, 0, 1, 1);
    printf("Adjacency Matrix Representation\n");
    displayMatrix(mat);
    floydWarshall(mat);

    //Dijkstra(mat, 0);
    return 0;
}
```